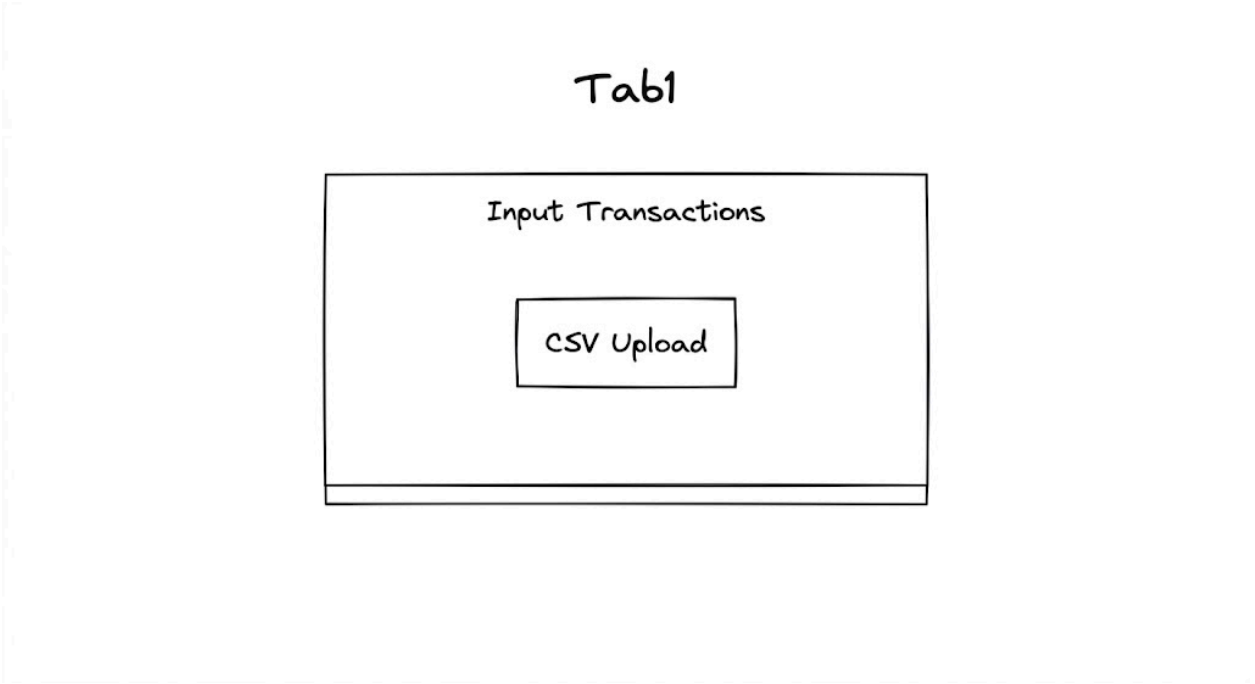
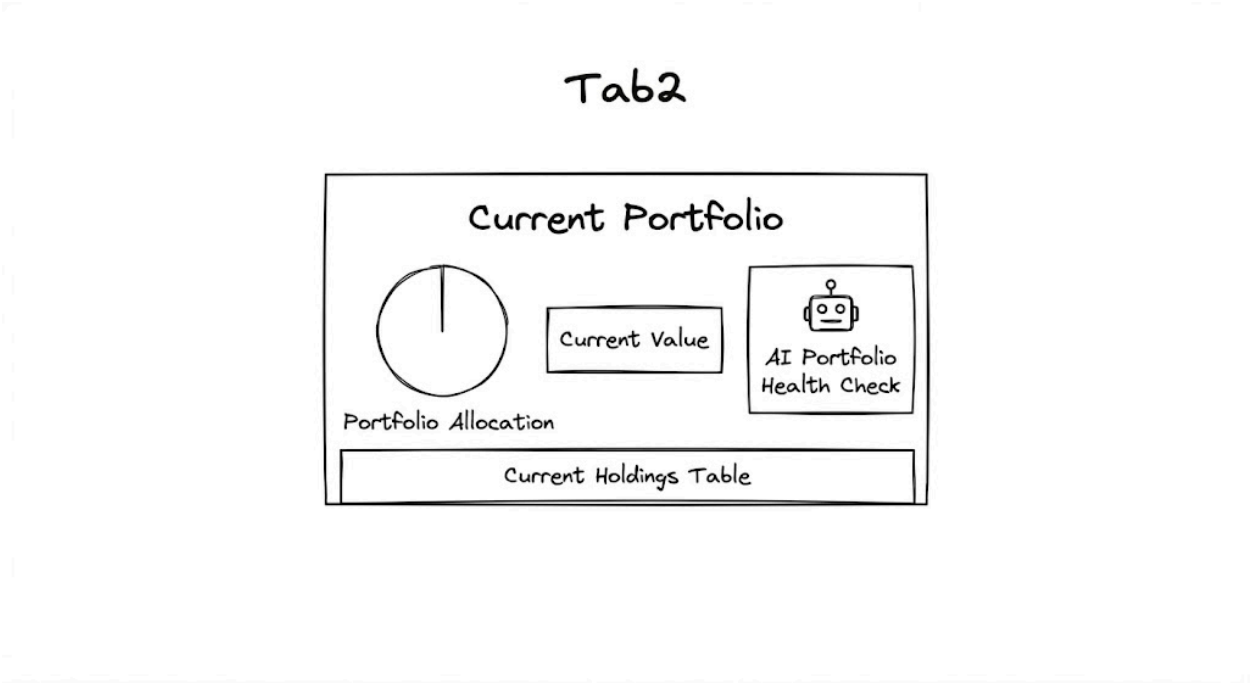
Project Outline

Visual overview of the application to be built

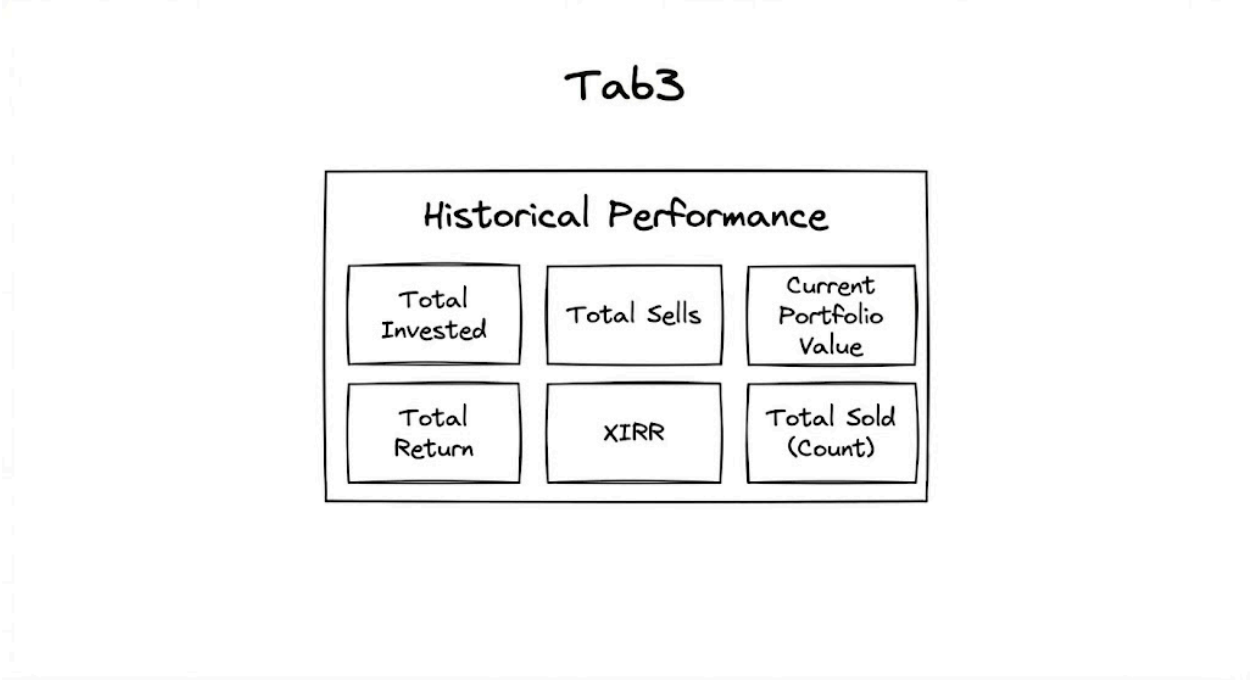
Tab 1:



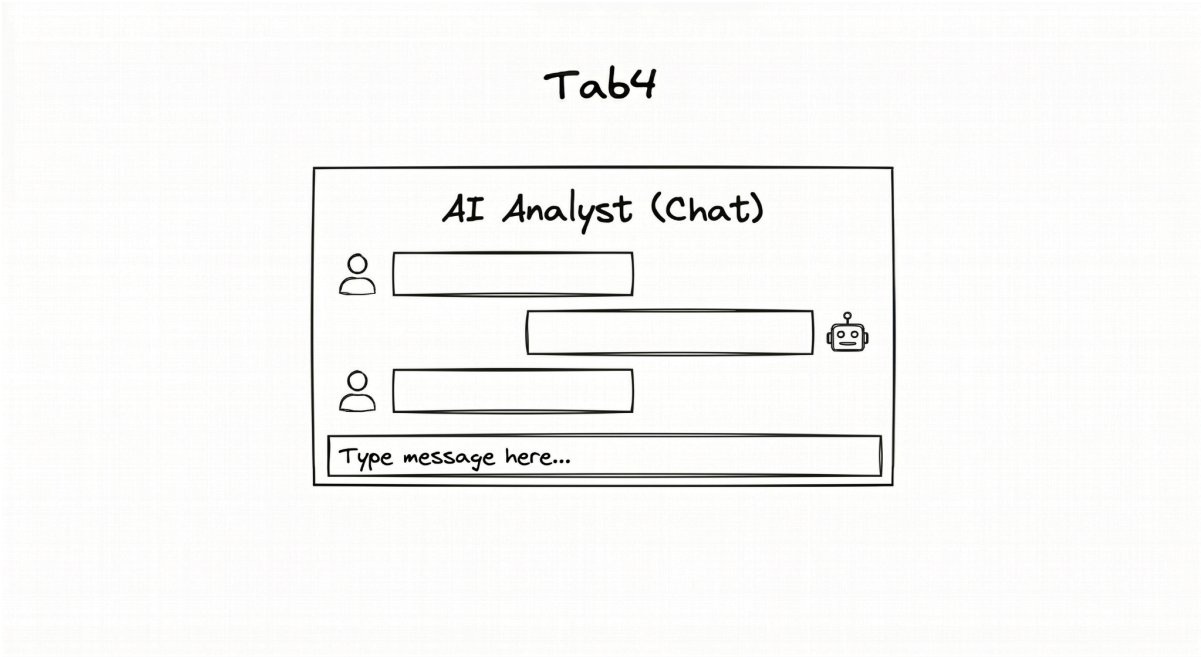
Tab 2:



Tab 3:


Tab3
Historical Performance
Total Invested
Total Sells
Current Portfolio Value
Total Return
XIRR
Total Sold (Count)

Tab 4:


Tab4
AI Analyst (Chat)
Type message here...

Sample Data & Demo

Sample Data Used: Sample Data

Click here to view the video demo.

Initial Prompt

Initial Prompt:

Build an AI-powered Streamlit app that acts as my personal Stock Analyst Agent to track and analyze my US stock portfolio. Use `yfinance` for live/historical stock data, `Groq` for LLM capabilities, and `uv` strictly for package management and virtual environment setup.

- Create 4 Tabs as follows: **Tab 1: Data Upload**
    - Takes a CSV upload of my stock transaction history (NO manual entry).
    - The CSV contains headers: `ticker, date, transaction_type (Buy/Sell), quantity, price`.
    - Display the uploaded dataframe.
- **Tab 2: Consolidated Portfolio View**
    - Pie chart showing portfolio allocation based on current stock value.
    - Metric card: Total Current Portfolio Value.
    - Table displaying stock-wise breakdown: Ticker Symbol, Current Quantity, Avg Cost basis (use FIFO method to account for sells), Current Price, Unrealized Profit/Gain (Value & %).
    - **AI Feature:** A small text container where Groq LLM provides a 2-3 sentence summary of portfolio health and concentration risk based on the current holdings.
- **Tab 3: Historical Performance**
    - Metric cards for: Total Investment (lifetime), Total Sells (lifetime proceeds), Current Portfolio Value, Total Return, and XIRR for the entire portfolio timeline.
- **Tab 4: AI Analyst (Chat)**
    - **AI Feature:** An interactive chat interface powered by Groq. Pass the portfolio history, performance metrics, and current holdings as context so I can ask questions like "Summarize my historical trading performance" or "Why is my portfolio down today?"
- Project Structure:
    - `app.py`: main application entry point
    - `utils/`: Folder for utility logic (`data_processing.py`, `portfolio_math.py`, `llm_agent.py`)
    - `components/`: Folder for each tab's UI logic

Refined Prompt

Refined Prompt:

You are an expert Python full-stack developer and AI engineer. Build a production-quality AI-powered Streamlit app that acts as my personal **US Stock Portfolio Analyst Agent**.

The app should allow me to upload my stock transaction history as a CSV, calculate my current portfolio holdings and performance, visualize allocation and returns, and provide AI-powered portfolio analysis using Groq.

Use the following stack strictly:

- Frontend/App Framework: Streamlit
- Stock Data: yfinance
- LLM Provider: Groq
- Package and virtual environment management: uv only
- Charts: Plotly or Streamlit-compatible charts
- Data processing: pandas, numpy
- Financial calculations: custom Python logic where required

Do not use manual stock entry. The portfolio must be built only from uploaded CSV transaction history.

Core Requirements

Create a Streamlit app with exactly 4 tabs:

1. Data Upload
2. Consolidated Portfolio View
3. Historical Performance
4. AI Analyst Chat

Input CSV Requirements

The app must accept only a CSV file upload containing the following headers:

ticker,date,transaction_type,quantity,price

Column definitions:

- `ticker`: US stock ticker symbol, for example AAPL, MSFT, NVDA
- `date`: transaction date
- `transaction_type`: Buy or Sell

- `quantity`: number of shares bought or sold
- `price`: transaction price per share

Validation requirements:

- Validate that all required columns exist.
- Validate that transaction_type contains only Buy or Sell, case-insensitive.
- Validate that quantity and price are positive numeric values.
- Validate that date can be parsed as a valid date.
- Normalize ticker symbols to uppercase.
- Show helpful Streamlit error messages if validation fails.
- Do not proceed to portfolio calculations until a valid CSV is uploaded.

Tab 1: Data Upload

Create a tab named **Data Upload**.

Features:

- Allow the user to upload a CSV file.
- Do not provide any manual entry form.
- Display the uploaded and cleaned dataframe.
- Show basic upload stats:
    - Number of transactions
    - Number of unique tickers
    - Date range of transactions
    - Number of Buy transactions
    - Number of Sell transactions
- Store the cleaned dataframe in Streamlit session state for use in the other tabs.

Tab 2: Consolidated Portfolio View

Create a tab named **Consolidated Portfolio View**.

This tab should calculate current holdings from the uploaded transaction history.

Use FIFO method for cost basis calculation:

- Buy transactions add lots.
- Sell transactions reduce lots starting from the oldest available buy lots.
- Remaining lots determine current quantity and remaining cost basis.
- Average cost basis = remaining cost basis / current quantity.
- If a stock is fully sold, it should not appear in current holdings.
- If a sell quantity exceeds available quantity, show an error.

Fetch current stock prices using yfinance.

For each current holding, calculate:

- Ticker Symbol
- Current Quantity
- Average Cost Basis using FIFO
- Current Price
- Current Market Value
- Unrealized Profit/Gain Value
- Unrealized Profit/Gain Percentage

Display:

1. Pie chart showing portfolio allocation based on current market value.
2. Metric card showing **Total Current Portfolio Value**.
3. Table showing the stock-wise breakdown with the following columns:
    - Ticker Symbol
    - Current Quantity
    - Avg Cost Basis
    - Current Price
    - Current Market Value
    - Unrealized Profit/Gain Value
    - Unrealized Profit/Gain %
4. AI summary text container:
    - Use Groq LLM to generate a 2-3 sentence summary of portfolio health.
    - The summary should mention concentration risk if one or a few stocks dominate the portfolio.
    - The summary should be based only on current holdings, allocation percentages, unrealized gains/losses, and total current value.
    - If Groq API key is missing, show a helpful warning instead of crashing.

Tab 3: Historical Performance

Create a tab named **Historical Performance**.

Calculate and display portfolio-level performance metrics.

Metric cards required:

1. Total Investment, lifetime
2. Total Sells, lifetime proceeds
3. Current Portfolio Value
4. Total Return
5. XIRR for the entire portfolio timeline

Definitions:

- Total Investment = total cash spent on all Buy transactions.
- Total Sells = total cash received from all Sell transactions.
- Current Portfolio Value = sum of current market value of all open holdings.
- Total Return = Total Sells + Current Portfolio Value - Total Investment.
- Total Return % = Total Return / Total Investment * 100.
- XIRR should use the full cashflow timeline:
    - Buy transactions are negative cashflows.
    - Sell transactions are positive cashflows.
    - Current portfolio value should be added as a final positive cashflow on today's date.
- Implement XIRR calculation robustly using scipy or a custom numerical method.
- Handle cases where XIRR cannot be calculated gracefully.

Also show:

- A historical transaction table.
- Optional cumulative cashflow or portfolio performance chart if feasible.

Tab 4: AI Analyst Chat

Create a tab named **AI Analyst Chat**.

Features:

- Build an interactive Streamlit chat interface.
- Use Groq as the LLM provider.
- Maintain chat history using Streamlit session state.
- Pass the following context to the LLM:
    - Uploaded transaction history
    - Current holdings
    - Portfolio performance metrics
    - Allocation percentages
    - Unrealized gains/losses
- The user should be able to ask questions such as:
    - "Summarize my historical trading performance."
    - "Why is my portfolio down today?"
    - "Which stocks are driving most of my gains?"
    - "Am I over-concentrated in any stock?"
    - "What are my worst-performing holdings?"
- The AI must not invent live market reasons unless supported by available data.
- If the user asks why a stock moved today, fetch recent stock price movement using yfinance before answering.

- The AI should include clear disclaimers that it is not providing financial advice.

Groq LLM Requirements

Use Groq for all AI features.

Requirements:

- Read the Groq API key from environment variable:

GROQ_API_KEY

- Do not hardcode API keys.
- Create a reusable Groq client wrapper in `utils/llm_agent.py`.
- Use a configurable model name, for example:

llama-3.1-8b-instant

or another available Groq model.

- The LLM response should be concise, practical, and grounded in the portfolio data.
- Include graceful error handling if:
    - API key is missing
    - Groq API call fails
    - LLM response is empty

Required Project Structure

Create the project using this structure:

```
stock-analyst-agent/
│
├── app.py
│
├── pyproject.toml
├── uv.lock
├── README.md
├── .env.example
├── .gitignore
│
├── utils/
│   ├── __init__.py
```

```
│   ├── data_processing.py
│   ├── portfolio_math.py
│   └── llm_agent.py
│
└── components/
    ├── __init__.py
    ├── data_upload_tab.py
    ├── portfolio_view_tab.py
    ├── historical_performance_tab.py
    └── ai_chat_tab.py
```

File Responsibilities

app.py

- Main Streamlit entry point.
- Configure page title, icon, and layout.
- Create the 4 tabs.
- Initialize session state.
- Call each tab component.

utils/data_processing.py

Handle:

- CSV validation
- Data cleaning
- Date parsing
- Ticker normalization
- Transaction sorting
- Error handling for invalid uploads

utils/portfolio_math.py

Handle:

- FIFO holdings calculation
- Average cost basis calculation
- Realized sell proceeds
- Current holdings calculation
- Total investment
- Total sells
- Current portfolio value

- Total return
- XIRR
- Allocation percentages
- yfinance price fetching
- Any required numerical calculations

utils/llm_agent.py

Handle:

- Groq client setup
- Prompt construction
- Portfolio summary generation
- Chat response generation
- Error handling for missing API key or API failure

components/data_upload_tab.py

Handle UI for:

- CSV upload
- Display uploaded dataframe
- Display upload stats

components/portfolio_view_tab.py

Handle UI for:

- Portfolio allocation pie chart
- Total current portfolio value metric card
- Current holdings table
- Groq-generated portfolio health summary

components/historical_performance_tab.py

Handle UI for:

- Historical performance metric cards
- Transaction table
- Optional performance chart

components/ai_chat_tab.py

Handle UI for:

- Chat interface
- Chat history
- Passing portfolio context to Groq
- Displaying AI responses

uv Requirements

Use uv strictly.

Provide setup instructions in README using uv only.

Include commands such as:

uv init
uv add streamlit pandas numpy yfinance plotly groq python-dotenv scipy
uv run streamlit run app.py

Do not use pip, venv, conda, poetry, or requirements.txt.

The project should rely on:

pyproject.toml
uv.lock

Environment Variables

Create `.env.example` with:

GROQ_API_KEY=your_groq_api_key_here
GROQ_MODEL=llama-3.1-8b-instant

Use `python-dotenv` to load environment variables locally.

Streamlit UI Requirements

The UI should be clean and easy to use.

Requirements:

- Use `st.set_page_config`.
- Use wide layout.

- Use tabs.
- Use metric cards.
- Use clear section headers.
- Use helpful info, warning, and error messages.
- Do not crash if no file is uploaded.
- Show a prompt asking the user to upload a CSV before showing portfolio tabs.
- Cache yfinance calls where appropriate using Streamlit caching.
- Format currency values as USD.
- Format percentages clearly.
- Round numerical outputs sensibly.

Data and Calculation Edge Cases

Handle these cases:

- Empty CSV
- Missing required columns
- Invalid dates
- Invalid transaction types
- Negative or zero quantity
- Negative or zero price
- Sell before any Buy
- Sell quantity greater than owned quantity
- Fully sold positions
- Failed yfinance price fetch
- Missing Groq API key
- XIRR not computable
- Duplicate transaction rows
- Mixed-case tickers and transaction types

AI Safety and Accuracy Requirements

The AI analyst should:

- Clearly state that outputs are informational and not financial advice.
- Avoid making buy/sell recommendations unless framed as general risk observations.
- Not hallucinate market news.
- Base portfolio analysis only on uploaded transactions, yfinance data, and computed metrics.
- Say when it does not have enough information.
- For “why is my portfolio down today?”:
    - Compare latest prices against previous close where possible.
    - Identify which holdings contributed most to the day’s move.

- Do not invent external news unless an external news API is explicitly added later.

README Requirements

Create a README.md containing:

1. Project overview
2. Features
3. Folder structure
4. CSV format example
5. Setup instructions using uv
6. Environment variable setup
7. How to run the app
8. Notes on FIFO cost basis
9. Notes on XIRR calculation
10. Disclaimer that the app is for educational/informational use only and not financial advice

Sample CSV

Include this sample CSV in the README:

ticker,date,transaction_type,quantity,price
AAPL,2023-01-10,Buy,10,145.00
MSFT,2023-02-15,Buy,5,250.00
AAPL,2023-06-20,Sell,3,180.00
NVDA,2023-09-01,Buy,4,470.00

Expected Final Output

Generate the complete working project with all files.

The final app should be runnable with:

uv run streamlit run app.py

Before finishing, check that:

- The project structure is correct.
- All imports work.
- The app runs without syntax errors.
- CSV upload works.

- FIFO calculations are correct.
- Current prices are fetched from yfinance.
- All 4 tabs render properly.
- Groq summary and chat work when `GROQ_API_KEY` is provided.
- The app handles missing API key gracefully.
- README contains uv-only setup instructions.

Commands

1. Initialize the virtual environment:
    uv venv

2. Activate the virtual environment:
    .venv\Scripts\activate

3. Install dependencies:
    uv pip install -r requirements.txt

4. Run the Streamlit application:
    streamlit run app.py

Sample Question:
    Summarize my historical trading performance